

Universidad Nacional de Mar del Plata
FACULTAD DE INGENIERÍA.
SISTEMAS OPERATIVOS

Segundo Parcial
7 de junio de 2024

Alumno: JUAN CRUZ MATEOS - ENTREGO 6 HOJAS + 2 DE EXAMEN

1	2	3	4	5	Puntos	Nota
3	3	2	2	2	12	
2,8	2,7	1,6	2	2	11,1	9,1

1. Conceptos generales (3 puntos; 0.5 cada inciso)

Defina brevemente los siguientes conceptos:

- Semáforo
- Dominio de Protección
- Seguridad
- Procesos Cooperativos
- Algoritmo del Banquero
- Monitor

2. Relación entre conceptos (3 puntos; 1 cada inciso)

- Defina procesamiento concurrente, sección crítica y exclusión mutua. Relacione estos conceptos.
- Defina Gestión de la Seguridad. Explique los tres planos de abordaje, relaciones y de un ejemplo de cada uno.
- Defina Deadlock. Mencione las cuatro condiciones que determinan que el sistema caiga en deadlock. Relacione y fundamente por qué podría ser una opción válida usar el algoritmo del avestruz.

3. Concurrencia (2 puntos)

Complete el pseudocódigo, utilizando semáforos e inicializando las variables que considere necesarias para implementar la lógica de una maquinaria que fabrica sillas industriales donde:

- 3 máquinas (PE) se encargan de fabricar la espalda del chaleco y colocarla en un recipiente compartido con capacidad máxima de 30 unidades.
- 2 máquinas (PFD) se encargan de fabricar el frente derecho en un recipiente compartido con una capacidad máxima de 15 unidades.
- 2 máquinas (PFI) se encargan de fabricar el frente izquierdo en un recipiente compartido con una capacidad máxima de 15 unidades.

- 1 máquina (PB) se encarga de fabricar botones en un recipiente compartido con una capacidad máxima de 100 unidades.
- Por último una máquina (PU) toma una espalda, 1 frente derecho, 1 frente izquierdo, 4 botones y deposita el chaleco armado en un espacio con capacidad ilimitada.

PE	PFD	PFI	PB	PE
While true do begin FabricaEspalda(); DepositEspalda(); end	While true do begin FabricaFrenteD(); DepositFrenteD(); end	While true do begin FabricaFrentel(); DepositFrentel(); end	While true do begin FabricaBoton(); DepositBoton(); end	While true do begin TomaEspalda(); TomaFrenteD(); TomaFrentel(); ArmaBase(); TomaBoton(); PegaBotones(); DepositChaleco(); end

4. Deadlocks (2 puntos)

Un Sistema Operativo utiliza el Algoritmo del Banquero como mecanismo para evitar deadlocks. Este sistema evalúa el estado en 3 oportunidades y se encuentra con el siguiente contexto:

- R1, R2, R3 y R4 son los recursos del sistema.
- El sistema cuenta con 3 unidades por cada recurso.
- P1, P2, P3 y P4 son procesos del sistema.
- La matriz de estado inicial del sistema es la siguiente:

Universidad Nacional de Mar del Plata
FACULTAD DE INGENIERÍA
SISTEMAS OPERATIVOS

	R1	R2	R3	R4
P1	0	0	0	1
P2	1	1	1	0
P3	1	1	0	0
P4	1	0	1	1

Determine si a partir de las siguientes matrices de necesidades máximas los siguientes son estados seguros o inseguros frente a la posibilidad de aparición de un deadlock. En caso de hallar un estado seguro, indicar la secuencia de ejecución de los procesos mostrando el estado de asignación de recursos en cada paso:

a)

	R1	R2	R3	R4
P1	2	3	1	2
P2	2	2	2	2
P3	1	1	2	1
P4	1	2	2	2

b)

	R1	R2	R3	R4
P1	0	1	3	3
P2	2	2	2	0
P3	3	1	1	0
P4	1	1	1	1

c)

	R1	R2	R3	R4
P1	1	2	2	1
P2	1	2	1	1
P3	1	1	1	1
P4	1	3	3	1

5. Protección (2 puntos)

En un sistema que se maneja con bits de permisos tipo Unix, Andrea es la dueña del archivo "script.py" y tenemos la siguiente configuración:

- Andrea: RW-
- Grupo "Profesores": R-X
- Todos: R--

Además, contamos con la siguiente información sobre los usuarios:

- Andrea es dueña del archivo y pertenece al grupo "Profesores"
- Beatriz pertenece al grupo "Profesores"
- César no pertenece a ningún grupo

Indique cuáles de las siguientes acciones son permitidas por el sistema:

Beatriz intenta leer la primera línea de script.py	✓
Beatriz intenta ejecutar script.py	✓
César intenta ejecutar script.py	X
César intenta escribir en script.py	X
Andrea intenta ejecutar script.py	X
Andrea intenta leer script.py	✓
Beatriz intenta agregar contenido al final de script.py	X
César intenta leer el último carácter de script.py	✓
Andrea intenta modificar los privilegios de script.py	✓

no puede porque
no tiene permisos X

1. a) Semaforos: un semaforo es una variable entera que se utiliza en la comunicación entre procesos para gestionar la exclusión mutua y la ~~sincronización~~ sincronización. Solo admite 3 operaciones atómicas (indivisibles):

- incr: inicialización con un valor no negativo
- decr: disminuye en 1 el contador. Si el valor resultante es negativo el proceso se bloquea y entra en la cola de espera del semaforo.
- signa: aumenta en 1 el contador. Si el valor resultante es mayor o igual a 0 se desbloquea un proceso en espera.

Se pueden implementar como semaforos generales (o contadores) como el de semáforo anteriormente, o de forma binaria permitiendo que la variable tome únicamente los valores 0 (bloqueado) o 1 (Libre).

b) Dominio de Protección: conjunto de procesos con iguales privilegios sobre los mismos objetos. Es volátil, no persistente y se crea con una relación 1:1 a partir de los roles.

c) Seguridad: significa ausencia de riesgos, teniendo presente que un nivel de seguridad del 100% no existe puesto que nunca puede garantizarse el 0% de riesgos. La seguridad es una medida de la confiabilidad de un ~~sistema~~ ^{sistema} dada por la sumatoria de los efectos potenciales de los mecanismos de seguridad implementados. El nivel de seguridad queda definido por la potencia de seguridad en función de los riesgos que se crean respecto a ~~causas~~ ^{causas} y los que no.

d) Procesos cooperativos: procesos ~~que~~ cuyos intervalos de ejecución se solapan, pudiendo estar concurrentes entre sí de la existencia del otro de 2 formas:

- indirecta: son procesos que comparten un recurso. Se trata de cooperación por compartimiento
- directa: son procesos que se comunican entre sí. Se trata de cooperación por comunicación

e) Algoritmo del Buzo:

estrategia de utilización o reposición de alocación de un recurso que establece que un recurso será asignado solo si el estado reciente luego de su asignación es seguro, es decir, si existe al menos 1 secuencia de desasignación de recursos que no conduzca a un deadlock. De lo contrario, la solicitud no es atendida, por haber un riesgo potencial futuro de deadlock.

el hacer esto b...

f) Monitor:

TDA utilizado en conurrencia que consta de variables internas, 1 o más procedimientos y una sucesión de inicialización. Cumple con las siguientes características:

- las variables internas solo pueden ser analizadas por los procedimientos del monitor
- un proceso entra al monitor cuando ejecuta 1 de sus procedimientos
- solamente 1 proceso puede estar dentro del monitor en un determinado instante. Cualquier otro proceso que quiera entrar será bloqueado.

En comparación con los Semáforos, poseen la misma funcionalidad pero son más fáciles de utilizar, a la par que facilitan la exclusión mutua (en los semáforos las programadoras deben administrar la exclusión mutua y la sincronización).

2.

a) El procesamiento concurrente tiene lugar cuando existen más de 1 proceso cuyos intervalos de ejecución se solapan. Puede ser real, cuando hay varios procesadores simultáneos, o aparente, cuando existe solo 1 procesador y los procesos toman turnos sobre él. En los procesos concurrentes pueden utilizar los recursos del sistema, y en caso de utilizar un recurso compartido tendrán una sección crítica: un segmento de código que manipula el recurso compartido y que debe ejecutarse de forma atómica. Por lo tanto, para que el resultado de la ejecución sea independiente de la ordenación de los procesos y para garantizar la atomicidad se requiere de exclusión mutua: solamente 1 proceso por vez es admitido en la sección crítica, quedando el resto bloqueado a la espera de su turno.

? el acceso al recurso?

b) La gestión de la seguridad es una tarea y responsabilidad exclusivamente humana y del profesional informático que consiste en definir, desarrollar e implementar el nivel de seguridad de un sistema como un resultado del análisis del mismo y el contexto en el que opera. Existen tres planes:

- Político: donde se definen los objetivos y prioridades. Responde al qué. En este plano se define el nivel de seguridad del sistema en función de los riesgos que se están dispuestos a correr. Existen 4 variables a considerar:

1. Costo a la política
2. Valor en juego
3. Efecto cuando ocurre un riesgo
4. Probabilidad de ocurrencia de un riesgo.

Tipicamente (1) es determinante en la decisión final que (2), (3) y (4) sugieran un nivel de seguridad mayor cuanto mayor son.

- Mecanismos: representan los medios e instrumentos que se utilizarán para cumplir con la política definida. Responde al cómo se va a cumplir la política.

Tipicamente existen N mecanismos para cumplir con una política (relación N-1). Pueden ser de distinta granularidad y de 2 tipos:

- a priori: preventivos
- a posteriori: reparadores

- Implementación: representan las acciones concretas que llevan adelante los mecanismos definidos. Es la materialización de los mecanismos en la realidad.

Falta ej.?

c) Un deadlock es una situación en un sistema de computación en la que un conjunto de procesos se encuentran bloqueados, que de ellos esperando por un recurso que otro proceso del conjunto posee. Esta situación resulta en que los procesos no avancen, representando fuerte y repetidamente en los logs del sistema:

- origen*
- $\chi \cdot \text{VAP} \rightarrow 0$
 - $\text{throughput} \rightarrow 0$
 - $\bar{T}_T \rightarrow \infty$
 - $\bar{T}_E \rightarrow \infty$
 - $\bar{T}_R \rightarrow \infty$

Para que una situación así pueda ocurrir deben darse 4 condiciones simultáneas, necesarias y suficientes para la existencia de deadlocks:

1. Exclusión mutua: solamente se permite que 1 proceso acceda a un recurso compartido en un instante determinado. ✓
2. Hold & wait: un proceso pueda solicitar un nuevo recurso mientras que mantiene los que ya le fueron asignados. ✓
3. ~~No Acquisición~~ No Acquisición: Los procesos deben liberar los recursos voluntariamente. El OS no puede arrebatar recursos de los procesos. ✓
4. Espera circular: situación en la que en un conjunto de procesos todos tienen asignado un recurso que otro solicita, generando un ciclo cerrado de asignación (solicita). ✓

Frente a los deadlocks se pueden tomar alguna de las siguientes posturas:

- Permitir que si caiga en un deadlock asignado siempre ~~se~~ que fuera posible, detectarlo y recuperarse al mismo, típicamente liberando procesos o arrebatar recursos. ✓
- Evitarlos mediante el algoritmo del banquero de forma tal ~~de~~ no asignar recursos si existe riesgo futuro de deadlock. ✓
- Prevenirlos: evitar en tiempo de diseño la posibilidad de ocurrencia de un deadlock al no cumplir con Hold & wait, o no cumplir con la condición de no arrebato, o con la condición de espera circular.
- Algoritmo del ovestruz: no hacer nada. Permitir la posibilidad de deadlocks delegando en otro (típicamente el usuario) su resolución.

Esta es una estrategia válida en sistemas donde la probabilidad de ocurrencia, el dolor en juego y el costo ante la ocurrencia de un deadlock son críticos. En el resto de los sistemas el ~~costo~~ overhead que resulta de los algoritmos de detección, evitación y recuperación es tal que no amerita su implementación. ✓

Ejercicio 3: Hoja 1 2 3
Declaraciones

3 PE : espaldas → recipiente compartido 30 un.
2 PFD : frasco de agua → " " 15 un
2 PFI : " " → " " 15 un
1 PB : botones → " " 100 un
1 PU : agua → 1E + 1F0 + 1F1 + 4 bot → chubasco → (limpio)

```
sem-t mutex-buffer-esp;      init (mutex-buffer-esp, 2);
sem-t mutex-buffer-fi;      init (mutex-buffer-fi, 2);
sem-t mutex-buffer-f1;      init (mutex-buffer-f1, 2);
sem-t mutex-buffer-bot;     init (mutex-buffer-bot, 1);
```

```
sem-t esp-llen0;             init (esp-llen0, 0);
"   esp-llen1;               init (esp-llen1, 30);
"   fi-llen0;                init (fi-llen0, 0);
"   fi-llen1;                init (fi-llen1, 15);
"   f1-llen0;                init (f1-llen0, 0);
"   f1-llen1;                init (f1-llen1, 15);
"   bot-llen0;               init (bot-llen0, 0);
"   bot-llen1;               init (bot-llen1, 100);
```

```
sem-t esp-llen0;             init (esp-llen0, 0);
"   esp-llen1;               init (esp-llen1, 30);
"   fi-llen0;                init (fi-llen0, 0);
"   fi-llen1;                init (fi-llen1, 15);
"   f1-llen0;                init (f1-llen0, 0);
"   f1-llen1;                init (f1-llen1, 15);
"   bot-llen0;               init (bot-llen0, 0);
"   bot-llen1;               init (bot-llen1, 100);
sem-t bot-llen;              init (bot-llen, 0);
```

```
Espaldas  espaldas [30];
FA        f2 [15];
FI        fi [15];
Botones   bot [100];
```

PE

~~it's a process,~~

```
while true:
    producer.epollin();
    wait(esp-uvw);
    wait(mutex.buffer-esp);
    Depositor.epollin();
    wait(mutex.buffer-esp);
    wait(mutex.buffer-esp);
    signal(mutex.buffer-esp);
    signal(mutex.buffer-esp);
    signal(esp-uvw);
```

PFD

```
while true:
    fabricarFD();
    wait(f2-uvw);
    wait(mutex.buffer-f2);
    Depositor.FabricaFD();
    signal(mutex.buffer-f2);
    signal(f2-uvw);
```

PFI

```
while true:
    fabricarFII();
    wait(f1-uvw);
    wait(mutex.buffer-f1);
    Depositor.FabricaFII();
    signal(mutex.buffer-f1);
    signal(f1-uvw);
```

PB

```
int cant;
while true:
    fabricarBoton();
    wait(buf-uvw);
    wait(mutex.buffer-buf);
    Depositor.Boton();
    cant = length(buf) % 4;
    if cant == 0 then signal(buf-uvw);
    signal(mutex.buffer-buf);
    signal(buf-uvw);
```

ejercicio 3: Hoja 2 y 3
PE, PFD, PFI, PB

Grupo 3: Hoja 3 2 3
PE

PE:

while true:

wait (esp. lleno);
wait (mutex-buffer-esp);
Toma Esp (1);
signal (mutex-buffer-esp);
signal (esp-vacio);

wait (f2. lleno);
wait (mutex-buffer-f2);
Toma FrenteD (1);
signal (mutex-buffer-f2);
signal (f2-vacio);

wait (f1. lleno);
wait (mutex-buffer-f1);
Toma FI (1);
signal (mutex-buffer-f1);
signal (f1-vacio);

or else case

wait (~~mutex~~-llen0);
wait (vacio-let);
wait (mutex-buffer-let);
Toma BORDAD (1);
signal (mutex-buffer-let);
signal (let-vacio);

pega burros

después de cada caso (1)

→ mismo buffer infinito conectado solo por PE

Podría usarse un buffer 184

4. R R₁ R₂ R₃ R₄
3 3 3 3

a)

max

	R ₁	R ₂	R ₃	R ₄
P ₁	2	3	1	2
P ₂	2	2	2	2
P ₃	1	1	2	1
P ₄	1	2	2	2

inicial

R ₁	R ₂	R ₃	R ₄
0	0	0	1
1	1	1	0
1	1	0	0
1	0	1	1

DIFF

R ₁	R ₂	R ₃	R ₄
2	3	1	1
1	1	1	2
0	0	2	1
0	2	1	1

D 0 1 1 1



a) ningún proceso podría continuar en base a las unidades disponibles.

∴ a) es un estado inseguro.

b)

max

	R ₁	R ₂	R ₃	R ₄
P ₁	0	1	3	3
P ₂	2	2	2	0
P ₃	3	1	1	0
P ₄	1	1	1	1

inicial

R ₁	R ₂	R ₃	R ₄
0	0	0	1
1	1	1	0
1	1	0	0
1	0	1	1

DIFF

R ₁	R ₂	R ₃	R ₄
0	1	3	2
1	1	1	0
2	0	1	0
0	1	0	0

D 0 1 1 1

✓ P₁ puede continuar.

	R ₁	R ₂	R ₃	R ₄
P ₁	0	1	3	3
P ₂	2	2	2	0
P ₃	3	1	1	0
P ₄				

R ₁	R ₂	R ₃	R ₄
0	0	0	1
1	1	1	0
1	1	0	0

R ₁	R ₂	R ₃	R ₄
0	1	3	2
1	1	1	0
2	0	1	0

D 1 1 2 2

✓ P₂ puede continuar.

	R ₁	R ₂	R ₃	R ₄
P ₁	0	1	3	3
P ₂				
P ₃	3	1	0	0
P ₄				

R ₁	R ₂	R ₃	R ₄
0	0	0	1
1	1	0	0

R ₁	R ₂	R ₃	R ₄
0	1	3	2
2	0	1	0

D 2 2 3 2

✓ P₁ puede cont.

	R ₁	R ₂	R ₃	R ₄
P ₁				
P ₂				
P ₃	3	1	0	0
P ₄				

R ₁	R ₂	R ₃	R ₄
1	1	0	0

R ₁	R ₂	R ₃	R ₄
2	0	1	0

D 2 2 3 3

✓ P₃ puede finalizar.

∴ b) es un estado seguro: secuencia segura = P₄, P₂, P₁, P₃

c)

	<u>MAX</u>			
	R_1	R_2	R_3	R_4
P_1	1	2	2	1
P_2	1	2	1	1
P_3	1	1	1	1
P_4	1	3	3	1

	<u>INIT</u>			
	R_1	R_2	R_3	R_4
	0	0	0	1
	1	1	1	0
	1	1	0	0
	1	0	1	1

	<u>DIF</u>			
	R_1	R_2	R_3	R_4
	1	2	2	0
	0	1	0	1
	0	0	1	1
	0	3	2	0

✓ P_2 puede finalizar

D 0 1 1 1

	R_1	R_2	R_3	R_4
P_1	1	2	2	1
P_2	—	—	—	—
P_3	1	1	1	1
P_4	1	3	3	1

	R_1	R_2	R_3	R_4
	0	0	0	1
	1	1	0	0
	1	0	1	1

	R_1	R_2	R_3	R_4
	1	2	2	0
	0	0	1	1
	0	3	2	0

✓ P_3 puede finalizar

D 1 2 2 1

	R_1	R_2	R_3	R_4
P_1	1	2	2	1
P_2	—	—	—	—
P_3	—	—	—	—
P_4	1	3	3	1

	R_1	R_2	R_3	R_4
	0	0	0	1
	1	1	0	0
	1	0	1	1

	R_1	R_2	R_3	R_4
	1	2	2	0
	—	—	—	—
	0	3	2	0

✓ P_4 puede finalizar

D 2 3 2 1

	R_1	R_2	R_3	R_4
P_1	1	2	2	1
P_2	—	—	—	—
P_3	—	—	—	—
P_4	—	—	—	—

	R_1	R_2	R_3	R_4
	0	0	0	1
	—	—	—	—
	—	—	—	—

	R_1	R_2	R_3	R_4
	1	2	2	0
	—	—	—	—
	—	—	—	—

✓ P_1 puede finalizar

D 3 3 3 2

∴ c) es un estado seguro: Secuencia segura = P_2, P_3, P_4, P_1

5. RESUELTO EN HOJA DE EXAMEN